

《数据库系统原理》大作业



系统设计报告

题目名称：含用户端、歌手端与管理端音乐平台

学号及姓名： 23373440 刘明昊

23373452 刘峻昊

23373532 程嘉烨



组内同学承担任务说明

承担任务	负责学生	备注	任务占比
系统服务器端开发	刘峻昊	数据库模式设计 后端开发 前后端接口设计 报告文档撰写	33.3%
系统功能设计与客户端开发	刘明昊	前端架构设计 歌曲功能设计 图表设计实现报告文档撰写	33.3%
系统客户端页面开发	程嘉烨	前端页面设计 用户功能设计 确立接口 实现报告文档撰写	33.3%

数据库系统原理-系统设计报告

一、需求分析

1.1 作业要求与系统实现

1、数据管理功能

- 基本要求：系统需要有数据管理功能，能够浏览、查看相关的关系表中的数据，并对其进行添加、修改或者删除。
- 用户端，歌手端可以查看歌曲，歌单，收藏等表的数据，以及歌曲、歌单等数据的增删改查
- 高级要求：（1）系统可以支持Excel或者XML格式文件数据的导入、导出。（2）支持高级用户对数据的审核。（3）支持数据的自动获取，比如从网站爬取相关数据
- 支持通过 Excel/XML 格式导入导出歌曲信息，新增歌曲需经管理员审核后正式上架，同时可对接音乐 API 自动抓取并同步歌曲相关信息，兼顾操作便捷性与信息准确性。

2、数据展示功能

- 基本要求：系统需要有系统展示功能，可以展示系统首页，数据的列表、详情，支持多种条件的查询，数据的排序、翻页、跳转等。可以采用图片、文字或者视频、音频的方式进行展示。
- 支持首页推荐、歌曲列表、歌单详情、搜索、排序、翻页，支持音乐播放
- 高级要求：支持对数据的全文检索，或者数据之间的比较、数据的推荐
- 支持全文检索（歌曲名，歌单名，歌手名）

3、业务功能

- 基本要求：系统需要支持某种业务功能，比如选课、购买商品、预定和入住旅馆房间等。
- 对于收费歌曲，用户需要购买才能播放歌曲
- 高级要求：支持相关人员对业务进行管理，比如对选课或者商品订单的审核，对快递小哥派发相关送货任务等。
- 管理员可以对歌单审核，然后修改歌单;可以对歌手注册审核;然后发放歌手资格

4、统计分析

- 基本要求:系统需要支持统计分析功能，支持按照不同的角度对数据进行统计汇总（比如按商品类别、价格区间、生产厂商等角度统计销售情况），用报表和统计图形展示统计结果，以及将其导出为PDF或者word文件。
- 支持对用户的歌曲播放历史生成总结报告并导出

- 高级要求：（1）可以对数据进行钻取操作，按照不同的粒度对数据进行统计，比如从省到其下属的市，从市到其下属的区县逐级对商品的产地进行统计分析。（2）数据的关联，比如从论文的信息关联到相关的科研人员，从科研人员关联到相关的单位进行统计分析。
- 支持钻取分析（如从歌手到歌曲），歌曲数据；关联分析（如歌曲与歌手），从歌曲可以关联到歌单，歌手的收藏情况

5、安全防护

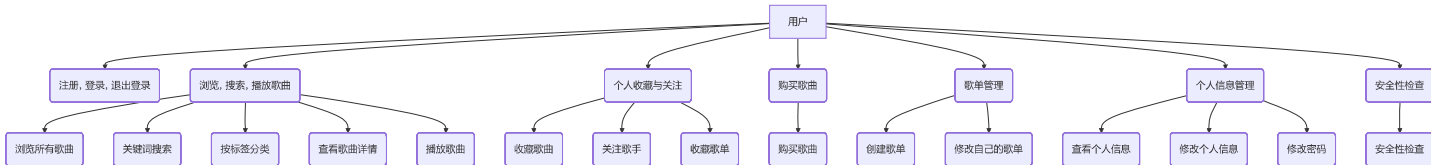
- 基本要求：系统需要有安全防护功能，支持多个类别的用户，提供用户的注册、审核、权限分配、登录等功能。
- 系统支持用户、歌手、管理员三个类别用户。支持用户和歌手的登陆注册功能。对于用户的审核与权限分配，管理员对歌曲上传与歌曲信息修改进行审核后，歌曲数据才展示在用户端，用户才能有权限查看，播放，购买。
- 高级要求：能够记录和获取用户的访问日志，甚至用户对于数据的操作，支持安全管理人员对日志进行浏览、查询和统计分析。
- 系统自动记录用户所有操作日志，涵盖访问行为、数据操作等关键信息；管理员可对日志进行浏览、多条件查询及统计分析，及时追溯操作轨迹，保障系统安全

1.2 系统需求综合描述

- 本次实验实现的音乐平台分为歌手端、用户端和管理员端。对于歌手、用户和管理员的功能设计如下：

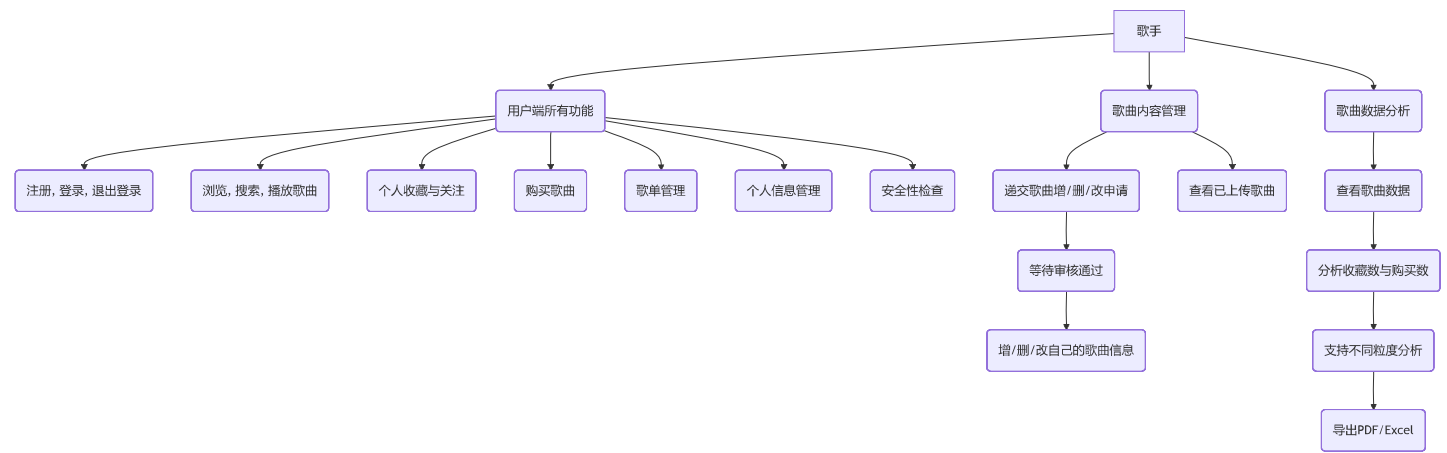
• 1. 用户端

- (1) 注册，登录，退出登录；
- (2) 浏览平台上所有歌曲，支持关键词搜索和按标签分类，查看歌曲详细信息，播放歌曲；
- (3) 收藏歌曲，关注歌手，收藏歌单；
- (4) 购买歌曲；
- (5) 创建歌单，修改自己的歌单；
- (6) 查看个人信息，修改个人信息和密码；
- (7) 安全性检查。



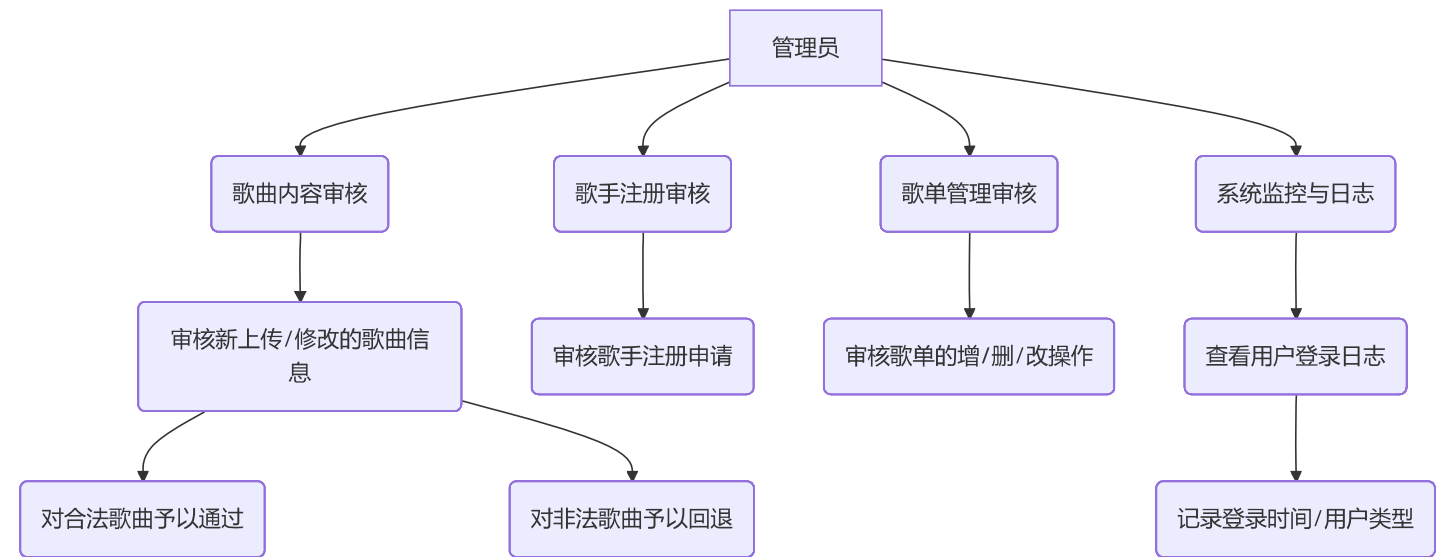
• 2. 歌手端：

- (1) 用户端所有功能
- (2) 增删改自己的歌曲信息，需要递交歌曲申请并等审核通过后可成功增删改歌曲；
- (3) 查看已上传的歌曲；
- (4) 查看歌曲数据，比如收藏数与购买数，支持不同粒度的分析和导出pdf、excel(比如单一指定歌曲，全部歌曲等)；

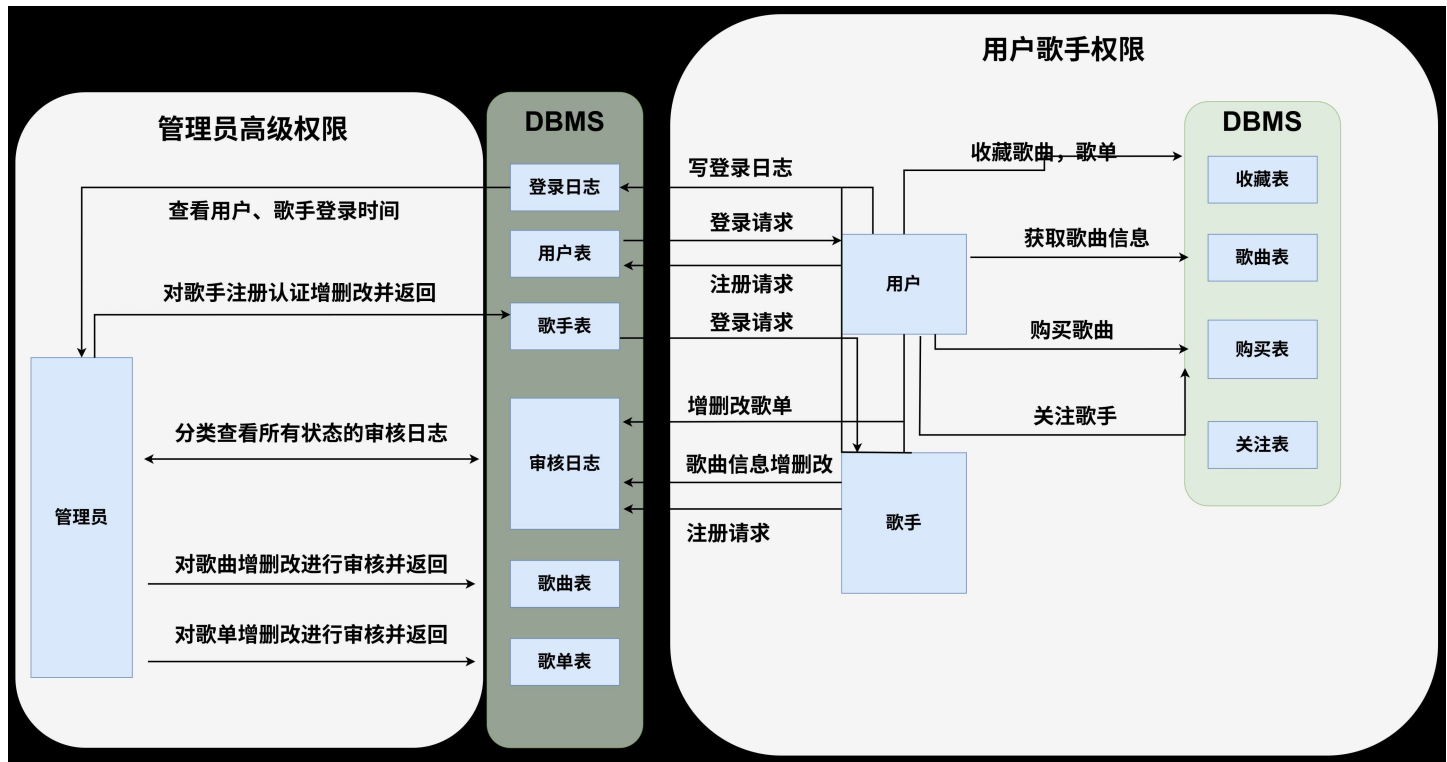


• 3. 管理员端:

- (1) 审核歌手上传的歌曲信息和修改的歌曲信息，对合法的歌曲上架请求予以审核通过，对非法歌曲、低俗恶臭歌曲等上架申请予以回退；
- (2) 查看用户的登录日志，记录用户每次的登录时间、用户类型并写入登录日志。
- (3) 对歌手注册申请审核
- (4) 对歌单增删改进行审核



1.3 数据流图



1.4 数据元素表

• 1、用户表

☐	☒ A= 名称	A= 数据类型	# 大小	☒ 是否必填	☒ 是否主键外键	A= 功能
1	user_id	Int	10	是	主键	用户id
2	user_password	Varchar	64	是	否	用户密码
3	user_name	Varchar	64	是	否	用户名
4	user_mobile	Char	11	是	否	用户电话号码
5	user_avatar	Image		否	否	用户头像
6	user_createtime	Date		是	否	用户注册时间
7	user_type	Int	4	是	否	区分歌手, 用户, 管理员

• 2、歌单表

☐	☒ A= 名称	A= 数据类型	# 大小	☒ 是否必填	A= 是否主键外键	A= 功能
1	playlist_id	Int	10	是	主键	歌单号
2	playlist_name	Varchar	64	是	否	歌单名
3	playlist_Image	Image		否	否	歌单封面
4	playlist_Introduction	Varchar	128	否	否	歌单介绍
5	playlist_userId	Int	10	是	外键(用户表)	歌单创建者

• 3、歌曲表

☐	☒ A= 名称	☒ 数据类型	# 大小	☒ 是否必填	☒ 是否主键外键	A= 功能
1	song_id	Int	10	是	主键	歌曲id
2	song_name	Varchar	100	是	否	歌曲名字
3	song_image	Image		否	否	歌曲封面
4	song_file_path	Varchar	100	是	否	歌曲音频文件地址
5	song_duration	Int	10	否	否	歌曲长度
6	song_price	Int	10	否	否	歌曲价格(0为免费)

• 4、歌单歌曲表

☐	☒ A≡ 名称	A≡ 数据类型	# 大小	A≡ 是否必填	A≡ 是否主键外键	A≡ 功能
1	playlist_songs_id	Int	10	是	主键	歌单歌曲关联id
2	playlist_id	Int	10	是	外键(歌单表)	歌单id
3	song_id	Int	10	是	外键(歌曲表)	歌曲id

• 5、用户收藏歌单表

☐	☒ A≡ 名称	A≡ 数据类型	# 大小	A≡ 是否必填	A≡ 是否主键外键	A≡ 功能
1	star_playlist_id	Int	10	是	主键	歌单收藏id
2	playlist_id	Int	10	是	外键(歌单表)	歌单id
3	user_id	Int	10	是	外键(用户表)	用户id

• 6、用户收藏歌曲表

☐	☒ A≡ 名称	A≡ 数据类型	# 大小	A≡ 是否必填	A≡ 是否主键外键	A≡ 功能
1	star_song_id	Int	10	是	主键	歌曲收藏id
2	song_id	Int	10	是	外键(歌曲表)	歌曲id
3	user_id	Int	10	是	外键(用户表)	用户id

• 7、用户购买歌曲表

☐	☒ A≡ 名称	A≡ 数据类型	# 大小	A≡ 是否必填	A≡ 是否主键外键	A≡ 功能
1	buy_song_id	Int	10	是	主键	歌曲购买id
2	song_id	Int	10	是	外键(歌曲表)	歌曲id
3	user_id	Int	10	是	外键(用户表)	用户id

• 8、用户关注表

☐	☒ A≡ 名称	A≡ 数据类型	# 大小	A≡ 是否必填	A≡ 是否主键外键	A≡ 功能
1	follow_id	Int	10	是	主键	关注id
2	follower_id	Int	10	是	外键(用户表)	关注者id
3	followed_id	Int	10	是	外键(用户表)	被关注者id

• 9、用户登录日志

☐	☒ A≡ 名称	A≡ 数据类型	# 大小	A≡ 是否必填	A≡ 是否主键外键	A≡ 功能
1	log_id	Int	10	是	主键	用户审核id
2	log_user_id	Int	10	是	外键	用户id
3	log_user_name	Varchar	64	是	否	用户名
4	log_user_type	Int	4	是	否	用户类型
5	log_time	Date		是	否	登录时间

• 10、歌单审核日志

☐	☒ A≡ 名称	⊙ 数据类型	# 大小	☺ 是否必填	☺ 是否主键外键	A≡ 功能
1	check_id	Int	10	是	主键	歌单审核id
2	check_playlist_id	Int	10	是	外键(歌单表)	歌单号
3	check_playlist_name	Varchar	64	是	否	歌单名
4	check_playlist_Image	Image		否	否	歌单封面
5	check_playlist_Introduct...	Varchar	128	否	否	歌单介绍
6	check_admit_time	Date		是	否	审核提交时间
7	check_modify_time	Date		否	否	审核更改时间
8	check_status	Varchar	10	是	否	审核状态

• 11、歌曲审核日志

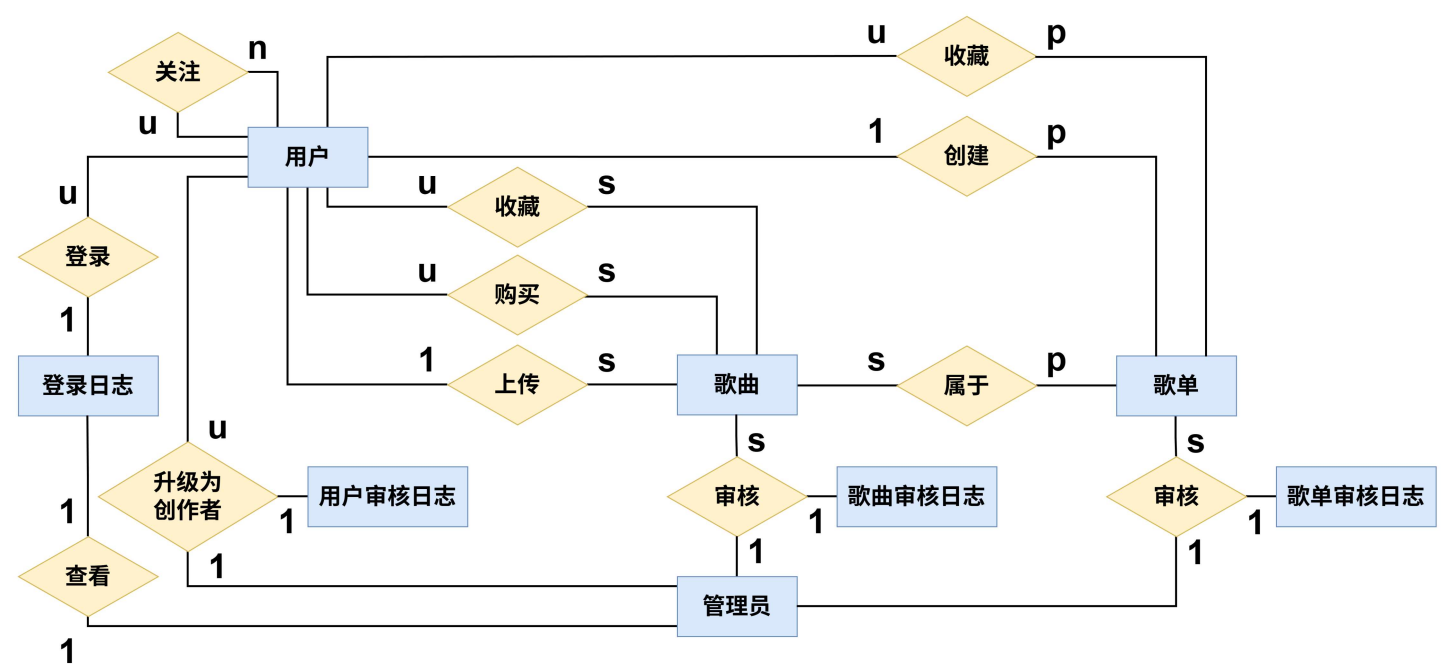
☐	☒ A= 名称	☒ 数据类型	# 大小	☒ 是否必填	☒ 是否主键外键	A= 功能
1	check_id	Int	10	是	主键	歌曲审核id
2	check_song_id	Int	10	是	外键(歌曲表)	歌曲id
3	check_song_name	Varchar	100	是	否	歌曲名字
4	check_song_image	Image		否	否	歌曲封面
5	check_song_file_path	Varchar	100	是	否	歌曲音频文件地址
6	check_song_duration	Int	10	否	否	歌曲长度
7	check_song_price	Int	10	否	否	歌曲价格(0为免费)
8	check_admit_time	Date		是	否	审核提交时间
9	check_modify_time	Date		否	否	审核更改时间
10	check_status	Varchar	10	是	否	审核状态

• 12、用户审核日志

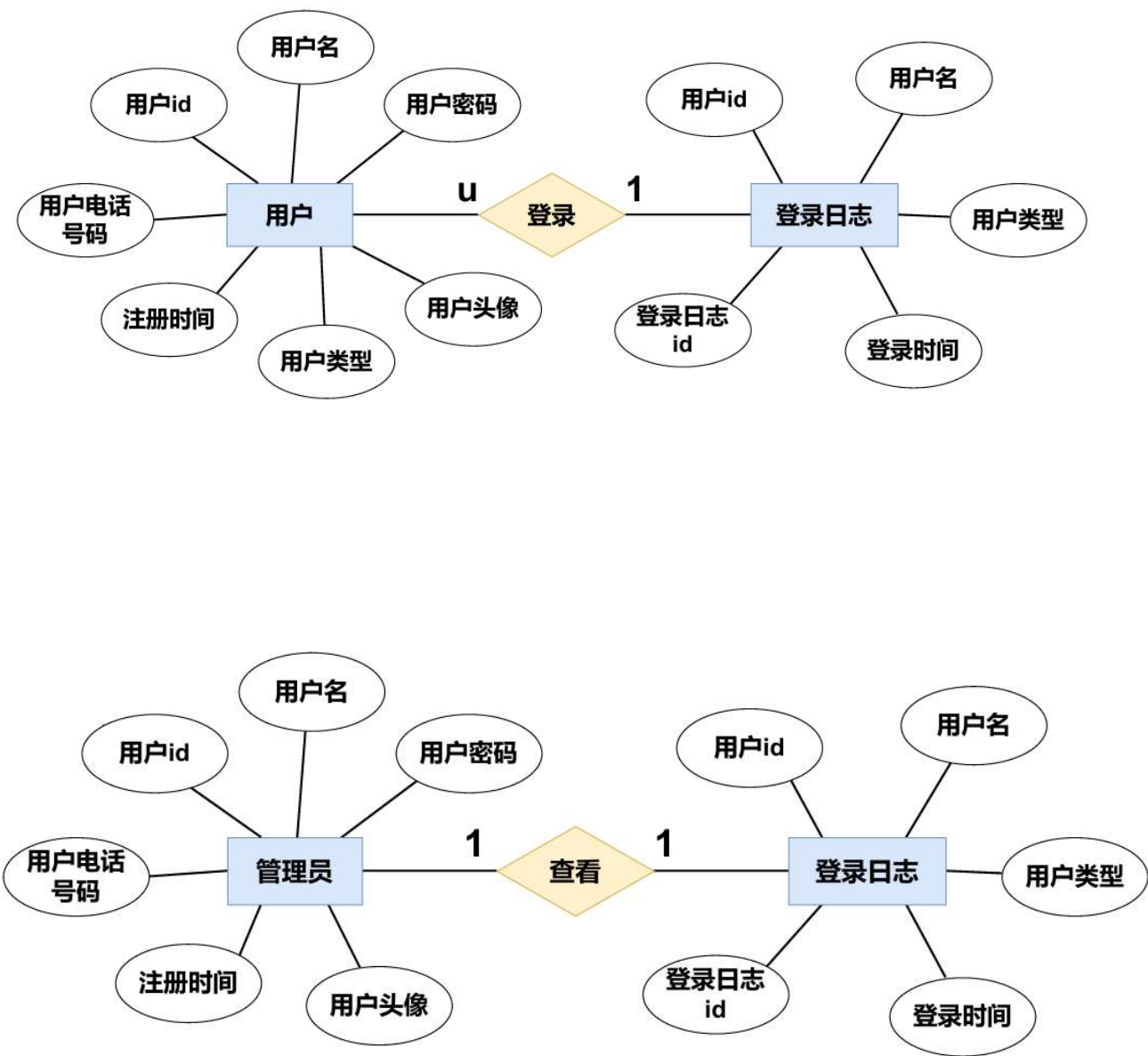
☐	☒ A= 名称	☒ 数据类型	# 大小	☒ 是否必填	☒ 是否主键外键	A= 功能
1	check_id	Int	10	是	主键	用户审核id
2	check_user_id	Int	10	是	外键	用户id
3	check_user_password	Varchar	64	是	否	用户密码
4	check_user_name	Varchar	64	是	否	用户名
5	check_user_mobile	Char	11	是	否	用户电话号码
6	check_user_avatar	Image		否	否	用户头像
7	check_user_type	Int	4	是	否	用户类型
8	check_admit_time	Date		是	否	审核提交时间
9	check_modify_time	Date		否	否	审核更改时间
10	check_status	Varchar	10	是	否	审核状态

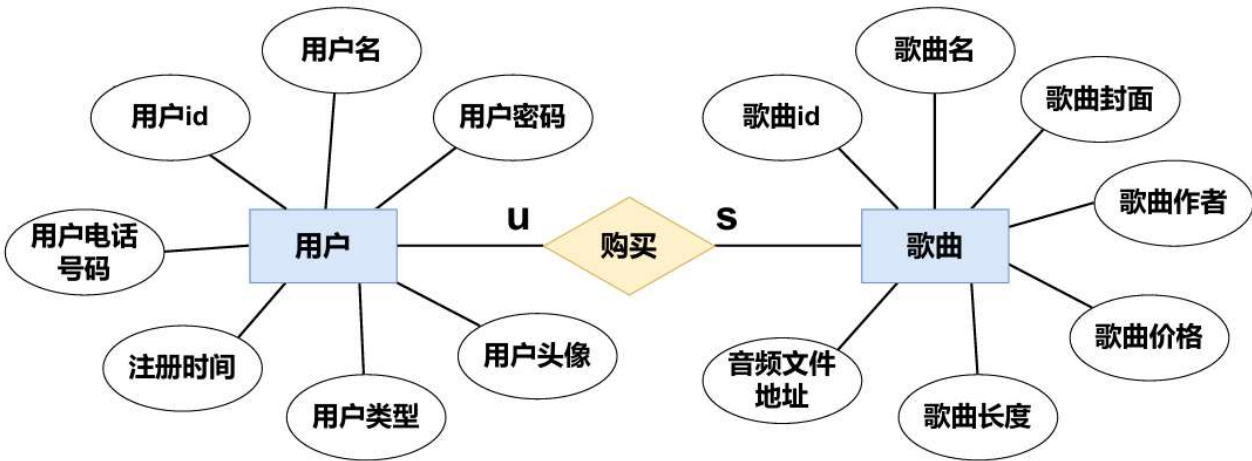
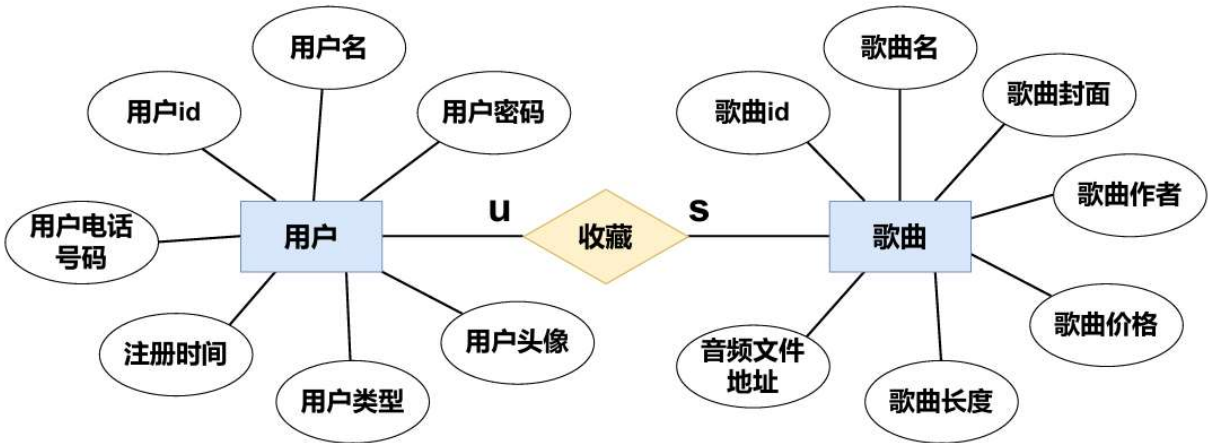
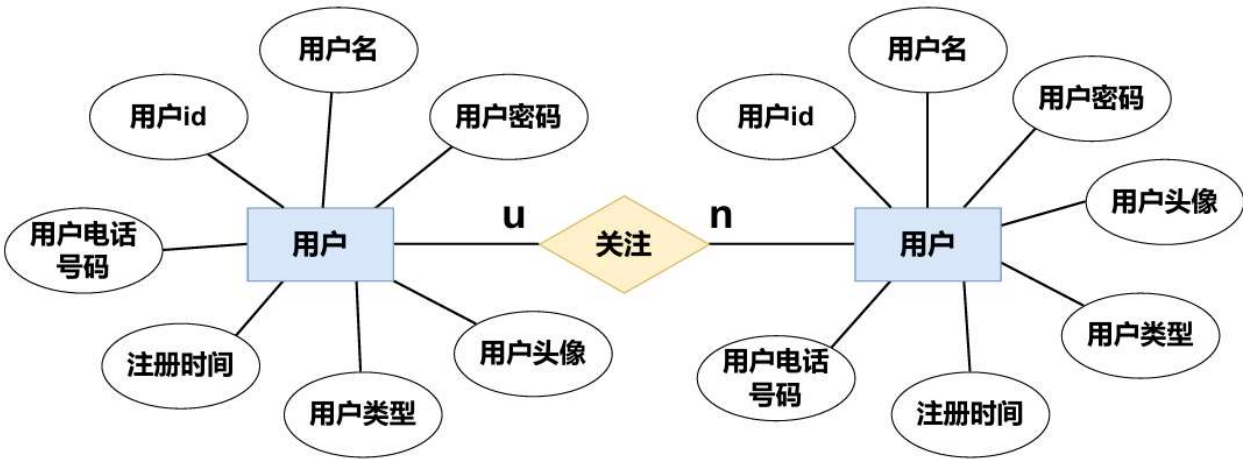
二、数据库概念模式设计

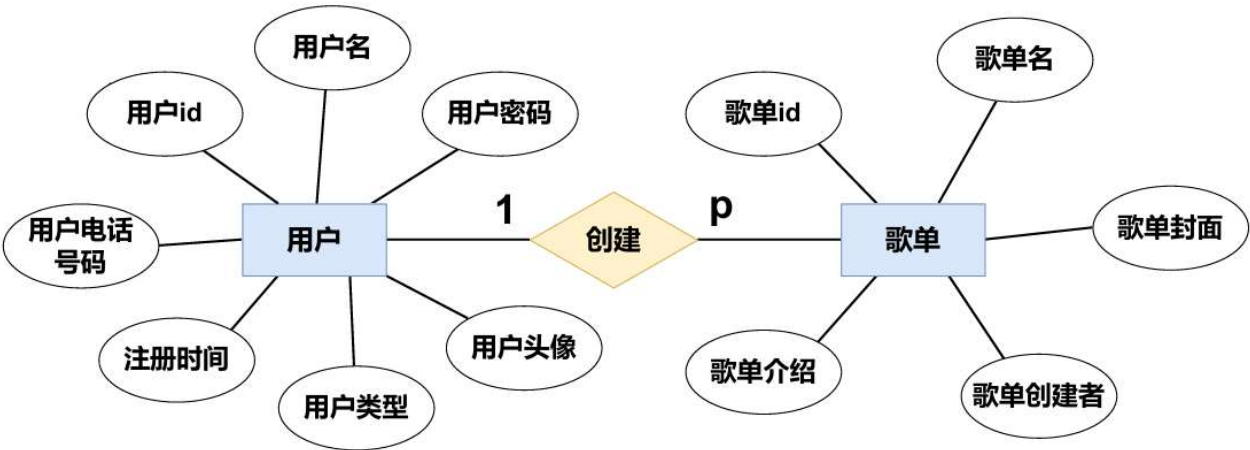
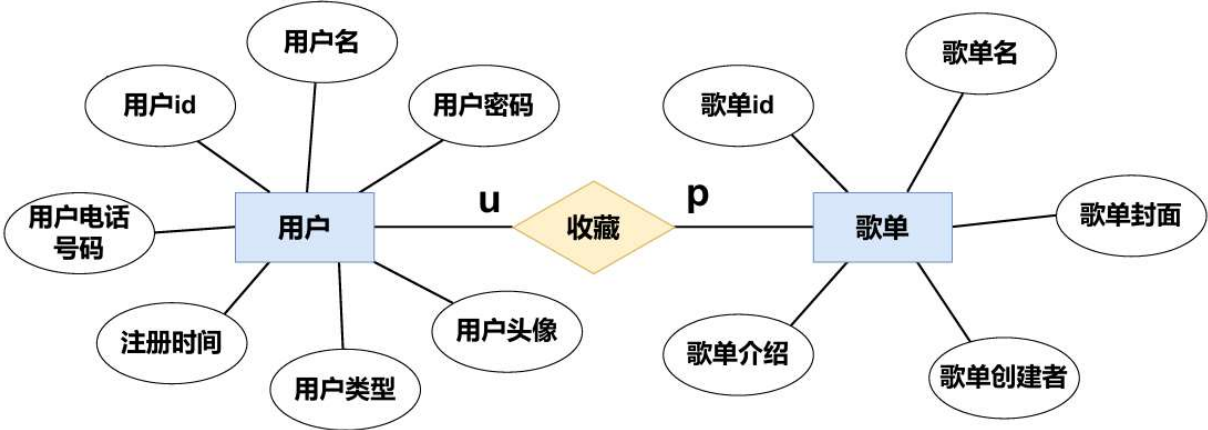
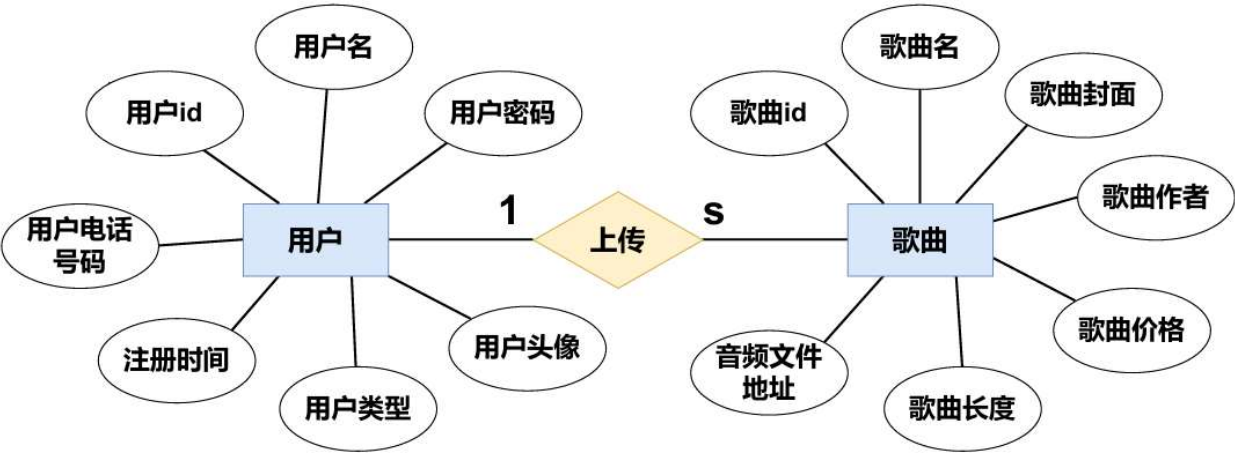
1、系统初步E-R图

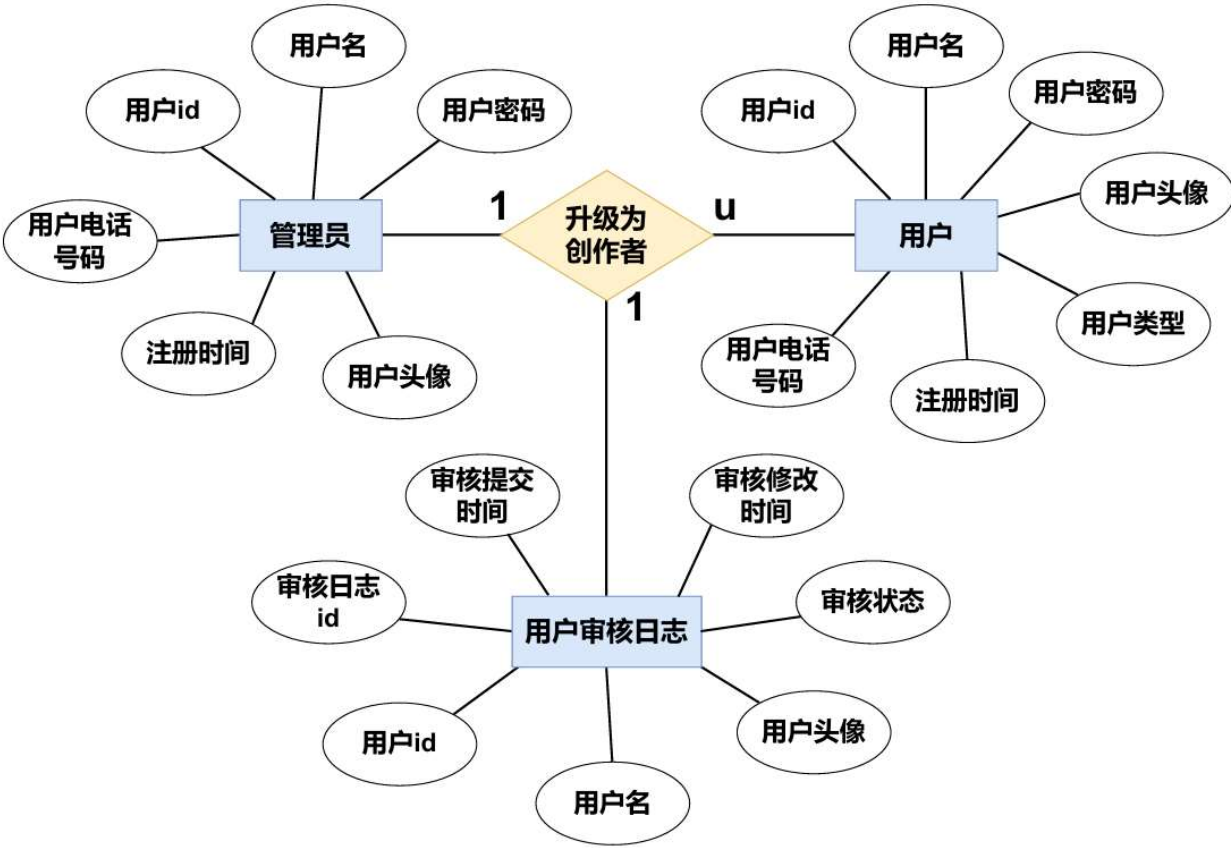
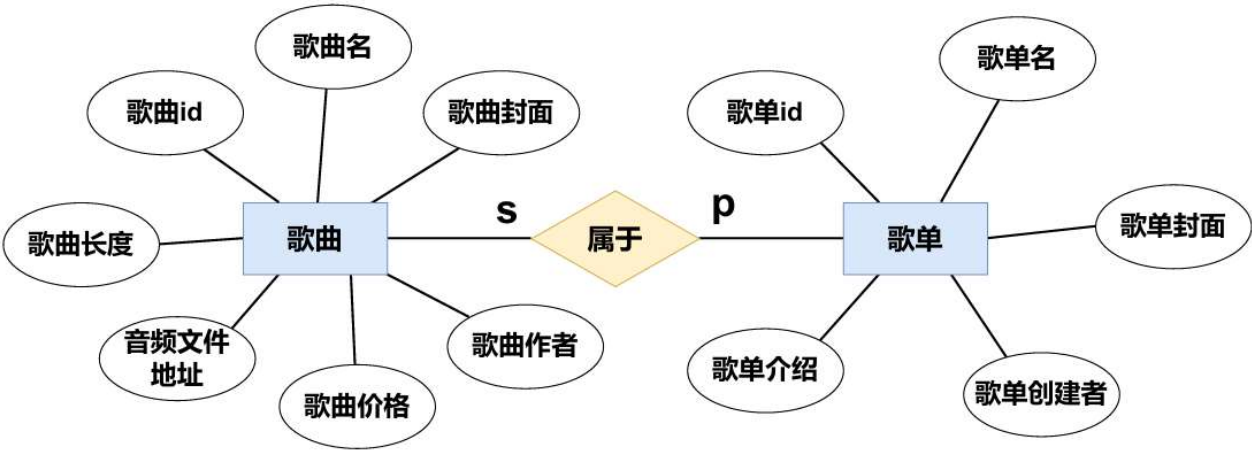


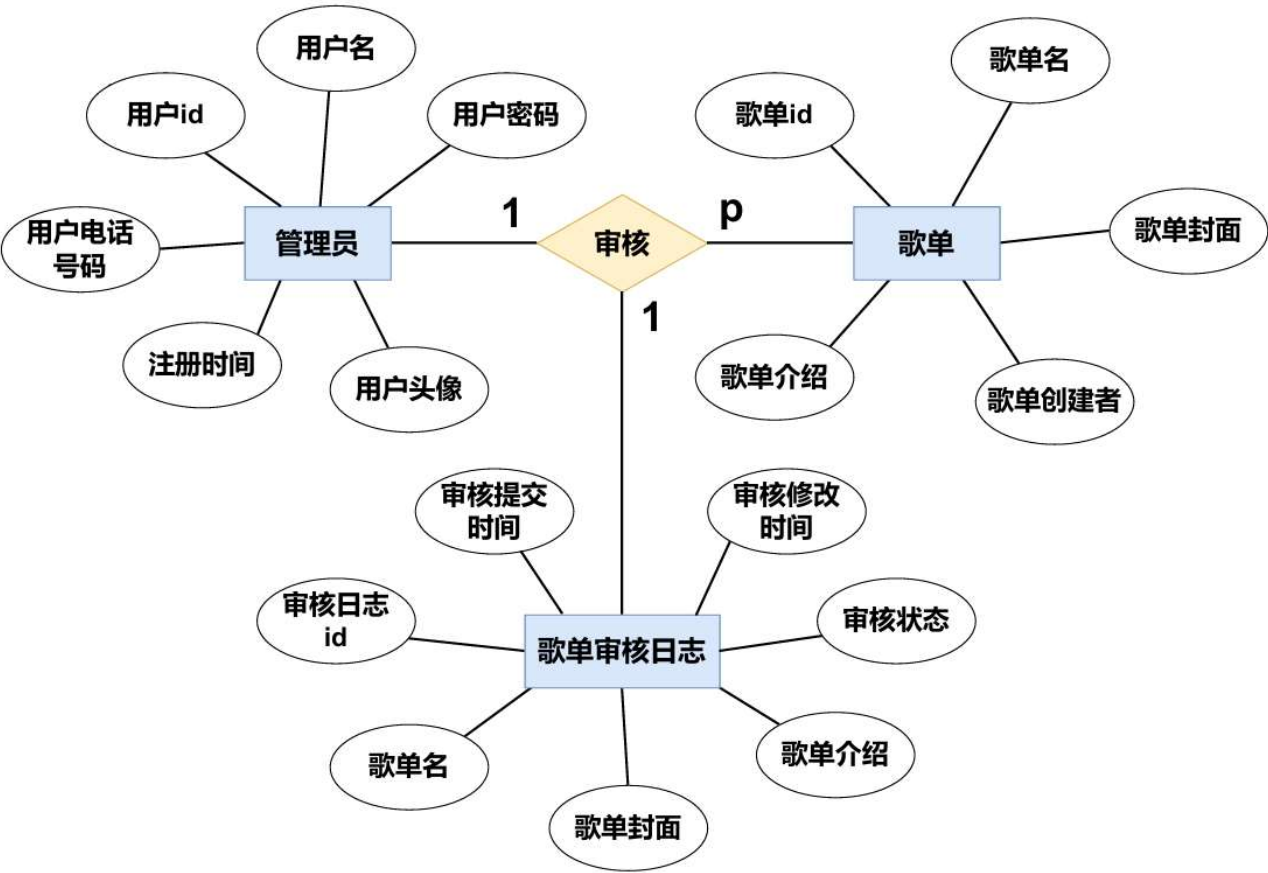
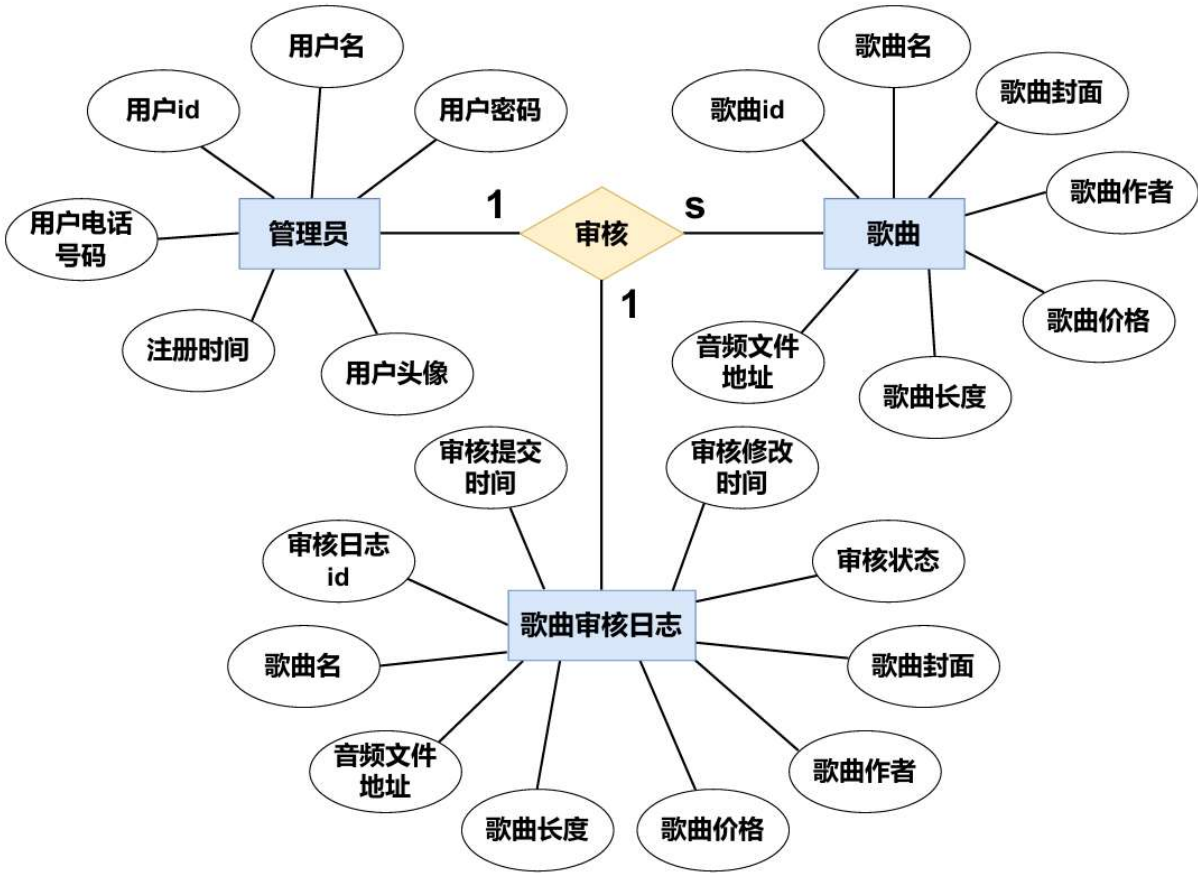
2、局部E-R图











三、数据库逻辑模式设计

1. 🎵 数据库关系模式：合并 E-R 图并规范化

设计过程中，我们采用自底向上的局部概念模式转化方法，先得到局部 E-R 图。为避免非全局概念模式导致的规范化不足及冗余数据问题，通过**逐步集成法**合并局部 E-R 图：

- **消除命名冲突**：建立命名对照表，统一id等字段命名，解决同名异义、异义同名问题；
- **消除属性冲突**：统一相同字段的长度、大小、是否可空等属性；
- **解决结构冲突**：根据关系类型确定采用属性或关系模式的结构，重复操作直至合并为全局概念模式，最终规范化得到以下关系模式：

编号	关系模式 (表名)	主键 (PK)	外键 (FK)	关键非主属性
1	用户表 (User)	user_id (用户ID)	无	用户密码, 用户名, 手机号码, 头像, 注册时间, 用户类型
2	歌单表 (Playlist)	playlist_id (歌单ID)	playlist_user_id (歌单创建者ID)	歌单名称, 歌单封面, 歌单介绍
3	歌曲表 (Song)	song_id (歌曲ID)	无	歌曲名称, 歌曲封面, 歌曲文件路径, 歌曲时长, 歌曲价格
4	歌单歌曲表 (PlaylistSong)	playlist_songs_id (歌单歌曲项ID)	playlist_id (歌单ID), song_id (歌曲ID)	无
5	用户收藏歌单表 (StarPlaylist)	star_playlist_id (收藏歌单ID)	playlist_id (歌单ID), user_id (用户ID)	无
6	用户收藏歌曲表 (StarSong)	star_song_id (收藏歌曲ID)	song_id (歌曲ID), user_id (用户ID)	无
7	用户购买歌曲表 (BuySong)	buy_song_id (购买记录ID)	song_id (歌曲ID), user_id (用户ID)	无

编号	关系模式 (表名)	主键 (PK)	外键 (FK)	关键非主属性
8	用户关注表 (Follow)	follow_id (关注ID)	follower_id (关注者ID), followed_id (被关注者ID)	无
9	用户登录日志 (LoginLog)	log_id (日志ID)	log_user_id (用户ID)	用户名, 用户类型, 登录时间
10	歌单审核日志 (CheckPlaylistLog)	check_id (审核ID)	check_playlist_id (歌单ID)	歌单名称, 歌单封面, 歌单介绍, 审核提交时间, 审核修改时间, 审核状态
11	歌曲审核日志 (CheckSongLog)	check_id (审核ID)	check_song_id (歌曲ID)	歌曲名称, 封面, 文件路径, 时长, 价格, 审核提交时间, 审核修改时间, 审核状态
12	用户审核日志 (CheckUserLog)	check_id (审核ID)	check_user_id (用户ID)	用户密码, 用户名, 手机号码, 头像, 用户类型, 审核提交时间, 审核修改时间, 审核状态

2. 验证关系模式范式等级规范化至 3NF

我们通过分析每个表中的函数依赖 (FD) 来验证其 3NF 规范化程度。

2.1 实体表

- **用户表:** user_id → (所有其他属性)
 - 所有非主属性都**完全依赖**于主键 user_id , 不存在部分依赖或传递依赖。 **满足 3NF。**
- **歌单表:** playlist_id → (所有其他属性)

- 所有非主属性都**完全依赖于**主键 `playlist_id` , 不存在部分依赖或传递依赖。**满足 3NF**。
- **歌曲表**: `song_id` $\rightarrow$ (所有其他属性)
 - 所有非主属性都**完全依赖于**主键 `song_id` , 不存在部分依赖或传递依赖。**满足 3NF**。

2.2 关系表

- **歌单歌曲表, 用户收藏歌单表, 用户收藏歌曲表, 用户购买歌曲表, 用户关注表**:
 - 这些表的主键 (`playlist_songs_id` , `star_playlist_id` , `star_song_id` , `buy_song_id` , `follow_id`) 是系统生成的**代理键**, 且表中**没有非主属性** (除了外键本身, 它们是主键的一部分或完全依赖于主键) 。
 - **FD**: 代理键 $\rightarrow$ (所有外键属性)
 - 仅存在平凡依赖, 不存在部分依赖或传递依赖。**满足 3NF**。

2.3 日志/审核表

日志表的关键在于**历史数据独立性**, 即日志记录的属性应仅依赖于日志本身的 ID。

- **用户登录日志** (`log_id` 是主键)
 - **FD**: `log_id` $\rightarrow$ (所有其他属性, 包括 `log_user_name` , `log_user_type`)
 - **说明**: `log_user_name` 和 `log_user_type` 可能会重复出现在 用户表 中, 但在日志表中, 它们记录的是**登录发生那一刻**的名称和类型, 仅依赖于 `log_id` 。
- **歌单审核日志, 歌曲审核日志, 用户审核日志** (以 `check_id` 为主键)
 - **FD**: `check_id` $\rightarrow$ (所有其他属性, 包括歌曲/歌单/用户的名称、图片、价格等历史信息)
 - **说明**: 与您商城案例的注释类似, 日志表中的**历史属性** (如 `check_song_name` 或 `check_playlist_name`) 记录的是**审核提交时**的数据快照, 它们仅依赖于 `check_id` , **不依赖于当前 歌曲表 或 歌单表 的主键**。这保证了审核记录的准确性。

根据对12个关系模式的分析: 该数据库关系模式中, 所有表的关系模式都不存在非主属性对码的**部分依赖或传递依赖**。因此, 该音乐数据库关系模式**满足 3NF 范式**。